TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

THAPATHALI CAMPUS

A Project Report

On

Heart Disease Prediction:
Using Logistic Regression and Artificial Neural Network

Submitted By:

Karun Lamsal (THA081BCT013)

Kishan Kumar Shah (THA081BCT014)

Submitted To:

Department of Electronics and Computer Engineering

Thapathali Campus

Kathmandu, Nepal

Under the Supervision of

Er. Suwarna Lingden

March, 2026

ACKNOWLEDGEMENT

We express our sincere thanks to the Institute of Engineering, Thapathali Campus, for giving us the chance to improve our knowledge and abilities through this project. We extend our sincere gratitude to the Department of Electronics and Computer Engineering for their ongoing assistance and for providing us with essential resources and information. We are particularly grateful to our supervisor, Er. Suwarna Lingden, for their essential guidance, support, and expert insights throughout this project's duration. Their guidance was vital for the successful fulfillment of our project.

We would also like to convey our gratitude to our colleagues and our supervisors for their support and encouragement. This project was more than just an academic task; it was an experience that enabled us to apply our practical skills in Data Science, particularly in Machine Learning and predictive analytics using Python. The educational experience and hands-on application have been incredibly advantageous, and we appreciate everyone who contributed to this learning journey.

Karun Lamsal (THA081BCT013)

Kishan Kumar Shah (THA081BCT014)

ABSTRACT

Heart disease is the leading cause of death worldwide, and early prediction using routine clinical data can significantly aid in timely intervention. In this project, we used the UCI Heart Disease dataset and built two predictive models - Logistic Regression and an Artificial Neural Network. After cleaning the data and applying feature standardization, both models were trained and compared. Logistic Regression, enhanced with cross-validation and regularization-based feature selection, slightly but consistently outperformed the ANN across most evaluation metrics. The findings suggest that simpler models can be equally or more effective than complex ones when working with small, low-dimensional datasets. This project applies the concepts learned in our Foundations of Data Science course.

Keywords: Data Science, Machine Learning, Heart Disease Prediction, Logistic Regression, Artificial Neural Network

Table of Content

ACKNOWLEDGEMENT ..... **I**

ABSTRACT ..... **II**

LIST OF FIGURES ..... **V**

1. INTRODUCTION **1**

1.1 Background 1

1.2 Motivation 1

1.3 Problem Statement 1

1.4 Objective 2

1.5 Scope and Limitations 2

2. LITERATURE REVIEW **3**

3. MATHEMATICAL FORMULATION **5**

3.1 Logistic Regression 5

3.2 Artificial Neural Network 7

3.3 Evaluation Metrics 9

4. DATASET DESCRIPTION AND COLLECTION METHOD **11**

4.1 Data Source and Origin 11

4.2 Data Collection Method 11

4.3 Feature Description 12

4.4 Target Variable Distribution 12

5. DATA CLEANING AND EXPLORATORY DATA ANALYSIS **13**

5.1 Initial Data Inspection 13

5.2 Identification of Invalid Values 13

5.3 Imputation Strategy 13

5.4 Exploratory Data Analysis 14

6. FEATURE ENGINEERING **16**

6.1 Feature-Target Separation 16

6.2 Train-Test Split 16
6.3 Feature Standardization 16
7. MODEL DEVELOPMENT 17
7.1 Logistic Regression 17
7.2 Artificial Neural Network 19
8. MODEL EVALUATION AND VALIDATION 21
8.1 Cross-Validation Strategy 21
8.2 Logistic Regression Results 21
8.3 ANN Results 22
9. RESULTS INTERPRETATION 23
9.1 Model Comparison 23
9.2 ROC Curve Analysis 23
9.3 Why Logistic Regression Outperformed the ANN 24
10. CONCLUSION 25
10.1 Summary of Findings 25
10.2 Limitations 25
10.3 Future Work 25
10.4 Ethical Considerations 26
REFERENCES 27

List of Figures

Fig 5-4-2 : Target Distribution 14
Fig 5-4-3 : Correlation Heatmap 15
Fig 7-1-1 : Feature Importance 18
Fig 7-2-1 : ANN Design 19
Fig 7-2-3 : Train Vs Validation 20
Fig 8-2 : LR Confusion Matrix 21
Fig 8-3 : ANN Confusion Matrix 22
Fig 9-2 : ROC Curve Comparison 23

1. INTRODUCTION

1.1 Background

Cardiovascular diseases (CVDs) are the number one cause of death globally, responsible for around 17.9 million deaths per year, which is roughly 32% of all deaths worldwide [1]. Out of these, about 85% are caused by heart attacks and strokes, and the problem is worse in low and middle-income countries, where more than three-quarters of these deaths happen. Diagnosing heart disease usually involves methods like angiography, stress tests, or echocardiography, but these procedures are expensive, take time, and are not easily available in many parts of the world. Because of this, there is a real need for simpler and cheaper screening methods that can work with the kind of data already collected during routine checkups.

1.2 Motivation

Machine learning makes it possible to train models on patient data collected from routine checkups like blood pressure, cholesterol levels, chest pain type, ECG results, and exercise heart rate. These models can help doctors identify at-risk patients early on without needing expensive tests. For this project, we used the UCI Heart Disease dataset, which was originally collected at the Cleveland Clinic by Robert Detrano and contains 302 patient records with 13 clinical features. Since the dataset is relatively small, it raised an interesting question for us: does a complex model perform better than a simpler one when the data is limited?

1.3 Problem Statement

Does a complex deep learning model (ANN) provide better predictive performance than a simpler traditional model (Logistic Regression) for heart prediction on a small clinical dataset?

- Common assumption: more complex models always produce better results
- This may not be for small, low-dimensional datasets
- Complex models tend to overfit; simpler models may generalize better outcomes.

1.4 Objective

- To preprocess the UCI Heart Disease dataset and build Logistic Regression and ANN models for heart disease prediction
- To evaluate and compare both models and identify the most important features.

1.5 Scope and Limitations

- Limited to binary classification (disease vs. no disease)
- Only the Cleveland subset (302 records) of the UCI Heart Disease dataset is used
- Original dataset has data from four hospitals; only Cleveland is well-documented and commonly used
- Multi-class classification (severity levels 0 - 4) is outside the scope

2. LITERATURE REVIEW

Several Heart disease predictions using machine learning has been a widely explored area in recent years, with most studies relying on the UCI Cleveland dataset as a standard benchmark. Several researchers have experimented with different algorithms and preprocessing strategies, and their findings provide useful context for the approach taken in this project.

Mohan et al. [2] compared multiple algorithms including Naive Bayes, Random Forest, SVM, and Logistic Regression on this dataset and achieved 88.7% accuracy using a hybrid approach that combined Random Forest with a linear model. Their work highlighted that not all 13 features contribute equally to prediction, and that proper feature selection plays a significant role in model performance. This observation aligns with the L1 regularization used in this project, which independently zeroed out several low-contribution features. Similarly, Ali et al. [3] evaluated six different algorithms on the same dataset and found that Logistic Regression remained competitive at approximately 82% accuracy, particularly due to the small dataset size and the roughly linear relationships between features and the target variable. Their conclusion supports a key finding of this project - that simpler models can match or outperform complex ones when the data is limited.

A common trend across Kaggle-based implementations [5] is that accuracies generally fall in the 78–88% range depending on the model and data split used. However, most of these notebooks skip proper data cleaning. The dataset, originally collected at the Cleveland Clinic by Robert Detrano and hosted on the UCI Machine Learning Repository [4], contains 303 patient records with 14 attributes. The Kaggle version [5] provides the data in a convenient CSV format but has a known data quality issue where missing values (originally marked as '?') were silently replaced with numeric values (0 for thal, 4 for ca) instead of being marked as NaN. Failing to detect and correct these values can introduce bias into the predictions, yet most publicly available analyses overlook this problem entirely.

For the theoretical foundations used in this project, we referred to educational resources by Krish Naik [6] for building the end-to-end ML pipeline, StatQuest [7] for understanding the mathematics behind logistic regression including the sigmoid function and maximum likelihood estimation, and 3Blue1Brown [8] for gaining intuition on neural network architecture and backpropagation. The implementations were built using scikit-learn [9] for Logistic Regression and Keras [10] for the ANN model.

Summary and Positioning

Source	Type	Method / Focus	Key Finding
Mohan et al. [2]	Journal (IEEE)	Hybrid RF + Linear	Feature selection important; 88.7% accuracy
Ali et al. [3]	Journal	LR, SVM, RF, etc.	LR competitive (~82%) on small datasets
Kaggle notebooks [5]	Community	Various ML models	78–88% accuracy; many skip proper data cleaning

How this project differs:

- Directly compares LR with ANN on this dataset (most implementations do not)
- Properly identifies and corrects invalid thal=0 and ca=4 values via median imputation (most overlook this)
- Uses GridSearchCV for systematic hyperparameter tuning rather than relying on default parameters).

3. MATHEMATICAL FORMULATION

3.1 Logistic Regression

- Supervised classification algorithm
- Models the probability of a binary outcome as a function of input features
- Despite the name "regression," it is used for classification

3.1.1 Linear Combination

$$z = w_1x_1 + w_2x_2 + \ldots + w_{13}x_{13} + b$$

- x_1–x_{13} = 13 input features
- w_1–w_{13} = learned weights (coefficients)
- b = bias (intercept) term
- Each weight represents its feature's contribution to the prediction

3.1.2 Sigmoid (Logistic) Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Output bounded between 0 and 1 → suitable for probability estimation
- Monotonically increasing - higher z → higher probability
- At z = 0: σ(0) = 0.5 → natural decision boundary
- Differentiable everywhere → enables gradient-based optimization
- If σ(z) >= 0.5 → predict class 1 (disease); otherwise, class 0 (no disease)
- Threshold of 0.5 is default, adjustable based on cost of FP vs FN

3.1.3 Loss Function - Binary Cross-Entropy

$$L = -\frac{1}{n}\sum[y_i \log(p_i) + (1 - y_i)\log(1 - p_i)]$$

- n = number of training samples
- $y_i \in \{0, 1\}$ = true label for sample i
- $\hat{p}_i$ = predicted probability for sample i
- When $y_i = 1$ and $\hat{p}_i \sim 1$ (correct, confident): loss ~ 0
- When $y_i = 1$ and $\hat{p}_i \sim 0$ (incorrect, confident): loss → ∞

- Same logic applies symmetrically for $y_i = 0$
- Heavily penalizes confident wrong predictions → drives models toward accurate probability estimates

3.1.4 Regularization

Purpose: prevent overfitting (memorizing training noise instead of learning generalizable patterns)

L1 Regularization (Lasso)

$$L_{total} = L + \lambda \cdot \sum_{j=1}^{m} |w_j|$$

- Adds sum of absolute weight values to loss
- Can drive individual weights exactly to zero → automatic feature selection → sparse model
- In this project, L1 zeroed out: age, trestbps, chol, fbs, restecg

L2 Regularization (Ridge)

$$L_{total} = L + \lambda \cdot \sum_{j=1}^{m} w_j^2$$

- Adds sum of squared weights to loss
- Shrinks all weights toward zero but never exactly to zero
- Distributes penalty evenly across features

Regularization parameter C:

$$C = \frac{1}{\lambda}$$

- Smaller C → stronger regularization (more penalty on large weights)
- GridSearchCV selected C=0.1 with L1 → relatively strong regularization with feature selection was optimal

3.1.5 Optimization

- Optimal weights found by minimizing L_total iteratively
- liblinear solver used - coordinate descent approach, efficient for small datasets, supports both L1 and L2

3.2 Artificial Neural Network

- Computational model inspired by biological neural networks
- Interconnected layers of artificial neurons processing information through weighted connections

3.2.1 Single Neuron

$$a = \text{activation}\left(\sum_{j=1}^{n} w_j x_j + b\right)$$

- x_j = inputs, w_j = learnable weights, b = bias
- Activation function introduces non-linearity

3.2.2 Network Architecture

- Layer 1: Dense(64, activation='relu') - Input shape: (13,)
- Layer 2: Dropout(0.3) - Regularization
- Layer 3: Dense(32, activation='relu') - Hidden layer
- Layer 4: Dense(1, activation='sigmoid') - Output layer

- Total trainable parameters: 3,009
- Narrowing structure (64 → 32 → 1) → progressive compression of learned representations
- Standard approach for small tabular datasets

3.2.3 Activation Functions

ReLU (Rectified Linear Unit) - Hidden Layers

$$f(x) = \max(0, x) = \{ x, x > 0;\ 0, x \leq 0 \}$$

- Introduces non-linearity; without it, multi-layer network = single linear transformation
- Computationally efficient
- Mitigates vanishing gradient problem in deeper networks

Sigmoid - Output Layer:

$$\sigma(z) = \frac{1}{1 + e^{\{-z\}}}$$

- Same function as in LR
- Maps output to [0, 1] → probability estimate for binary classification

3.2.4 Loss Function

- Same binary cross-entropy as LR (Section 3.1.3)
- Standard loss for binary classification with neural networks

3.2.5 Backpropagation and Gradient Descent

Two-phase learning process:

- **Forward pass**: Input flows through the network layer by layer → produces prediction
- **Backward pass (backpropagation)**: Gradient of loss w.r.t. each weight computed via chain rule, propagated backwards from output to input

Weight update rule:

$$w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$$

- α = learning rate
- ∂L/∂w = partial derivative of loss w.r.t. weight (how much the weight contributed to error)

3.2.6 Adam Optimizer

- Adaptive Moment Estimation - adapts learning rate per weight individually
- Combines two techniques:
  - **Momentum**: exponentially decaying average of past gradients → smooth, consistent direction
  - **RMSProp**: exponentially decaying average of past squared gradients → scales learning rate by gradient magnitude
- Most widely used optimizer; robust with sparse/noisy gradients

3.2.7 Dropout Regularization

- During each training iteration, a random 30% of neurons are "dropped" (outputs set to zero)
- Prevents co-dependence between specific neurons → forces robust, distributed representations
- Effectively trains a different sub-network at each step → final model ~ ensemble average

3.3 Evaluation Metrics

Accuracy:

Accuracy = (TP + TN) / (TP + TN + FP + FN)

- Overall correctness: reasonable here since classes are approximately balanced (54/46)

Precision:

Precision = TP / (TP + FP)

- Of predicted disease cases, proportion that has disease
- High precision → fewer false alarms

Recall (Sensitivity):

Recall = TP / (TP + FN)

- Of actual disease cases, proportion correctly identified
- Most critical metric in medical context: FN (missed disease) has serious consequences

F1-Score:

F1 = 2 × (Precision × Recall) / (Precision + Recall)

- Harmonic mean of precision and recall → penalizes extreme imbalance between the two

AUC-ROC:

- ROC curve: TPR vs FPR at all classification thresholds
- AUC = 1.0 → perfect; AUC = 0.5 → random; AUC < 0.5 → worse than random
- Threshold-independence → more robust than accuracy for model comparison

4. DATASET DESCRIPTION AND COLLECTION METHOD

4.1 Data Source and Origin

- **Source**: Cleveland subset of the UCI Heart Disease dataset [3]
- **Collected by**: Robert Detrano, M.D., Ph.D., at V.A. Medical Center, Long Beach & Cleveland Clinic Foundation
- **Repository**: UCI Machine Learning Repository (University of California, Irvine)
- **Downloaded from**: Kaggle [5] in CSV format; 302 records (1 apparently lost during format conversion)

4.2 Data Collection Method

- All records collected from patients who underwent **coronary angiography** - invasive procedure using contrast dye and X-ray to visualize arterial blockages
- Angiography results = ground truth (target variable) for coronary artery disease
- 13 clinical features collected from **routine, non-invasive examinations** - standard cardiac checkup tests
- **Significance**: if the model works, it could screen patients using only routine test results → potentially reduce need for invasive angiography

4.3 Feature Description

#	Feature	Full Name	Description	Type	Range
1	age	Age	Patient age in years	Continuous	29–77
2	sex	Sex	1 = male, 0 = female	Binary	0, 1
3	cp	Chest Pain Type	0 = typical angina, 1 = atypical, 2 = non-anginal, 3 = asymptomatic	Categorical	0–3
4	trestbps	Resting Blood Pressure	mm Hg on admission	Continuous	94–200
5	chol	Serum Cholesterol	mg/dl	Continuous	126–564
6	fbs	Fasting Blood Sugar	>120 mg/dl (1 = true, 0 = false)	Binary	0, 1
7	restecg	Resting ECG	0 = normal, 1 = ST-T abnormality, 2 = LV hypertrophy	Categorical	0, 1, 2
8	thalach	Max Heart Rate	Maximum during exercise stress test	Continuous	71–202
9	exang	Exercise-Induced Angina	1 = yes, 0 = no	Binary	0, 1
10	oldpeak	ST Depression	Exercise-induced relative to rest	Continuous	0–6.2
11	slope	ST Slope	0 = upsloping, 1 = flat, 2 = down sloping	Categorical	0, 1, 2
12	ca	Major Vessels	Vessels colored by fluoroscopy	Ordinal	0–3
13	thal	Thalassemia	1 = normal, 2 = fixed defect, 3 = reversible defect	Categorical	1, 2, 3
14	**target**	**Diagnosis**	**1 = disease, 0 = no disease**	**Binary**	**0, 1**

4.4 Target Variable Distribution

- Heart disease present (target = 1): 164 patients (54.3%)
- No heart disease (target = 0): 138 patients (45.7%)
- Approximately balanced (54:46 ratio) → class imbalance not a concern
- SMOTE or class weighting not necessary

.

5. DATA CLEANING AND EXPLORATORY DATA ANALYSIS

5.1 Initial Data Inspection

- ***df.info()*** → 302 rows, 14 columns, all numeric (int64/float64)
- ***df.isnull().sum()*** → no NaN values in any column
- ***df.duplicated().sum()*** → no duplicate rows
- However, absence of NaN ≠ data quality guaranteed
- Column value ranges must be examined individually for invalid entries

5.2 Identification of Invalid Values

Examined via ***df['column'].value_counts():***

Issue 1: thal = 0 (2 rows)

- Valid values: 1 (normal), 2 (fixed defect), 3 (reversible defect)
- thal = 0 has no valid medical interpretation
- Root cause: originally '?' (unknown) in UCI files → converted to 0 in Kaggle CSV
- Silent data corruption - numeric value does not trigger null-checking

Issue 2: ca = 4 (4 rows)

- Valid values: 0, 1, 2, 3
- ca = 4 is outside documented valid range
- Same root cause - '?' in original UCI files → converted to 4

5.3 Imputation Strategy

Strategy	Pros	Cons
Row deletion	Simple	Data loss (302 → 296)
Mode imputation	Common method	Inflates dominant value (ca=0, 57.9%)
Median imputation	Keeps all rows, less skew	--

- Mean imputation not appropriate: thal is categorical, ca is ordinal → fractional values meaningless

- **Decision**: Median imputation
  - thal: median of {1, 2, 3} = 2
  - ca: median of {0, 1, 2, 3} = 0
- All 302 rows presence

5.4 Exploratory Data Analysis

5.4.1 Summary Statistics

- Age: 29–77 years, mean ~ 54
- Cholesterol: 126–564 mg/dl, mean ~ 246
- Max heart rate (thalach): 71–202 bpm, mean ~ 150
- Sex distribution: 207 male (68.5%), 96 female (31.5%)

5.4.2 Target Distribution

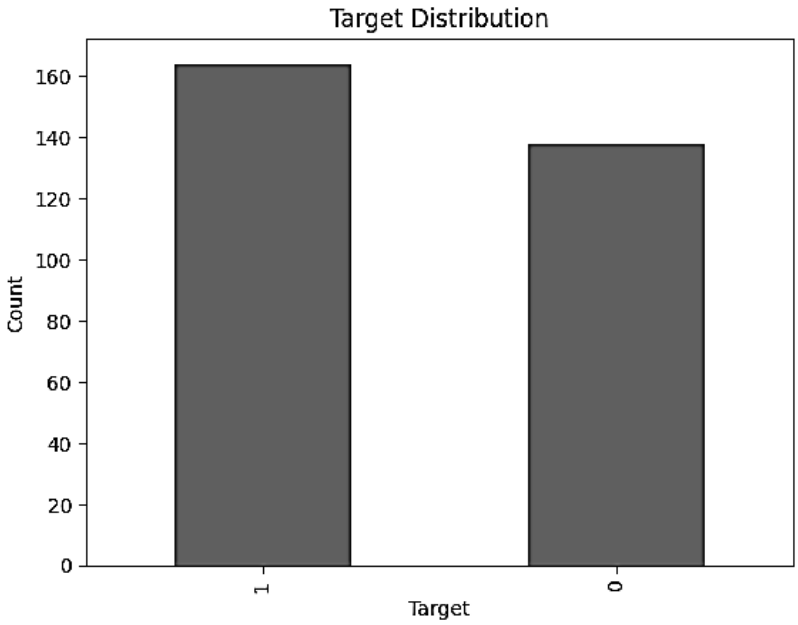

Fig 5-4-2 : Target Distribution

- 164 disease (54.3%) / 138 healthy (45.7%)
- Baseline accuracy (majority class prediction) = 54.3%
- Any useful model must substantially exceed this baseline

5.4.3 Correlation Analysis

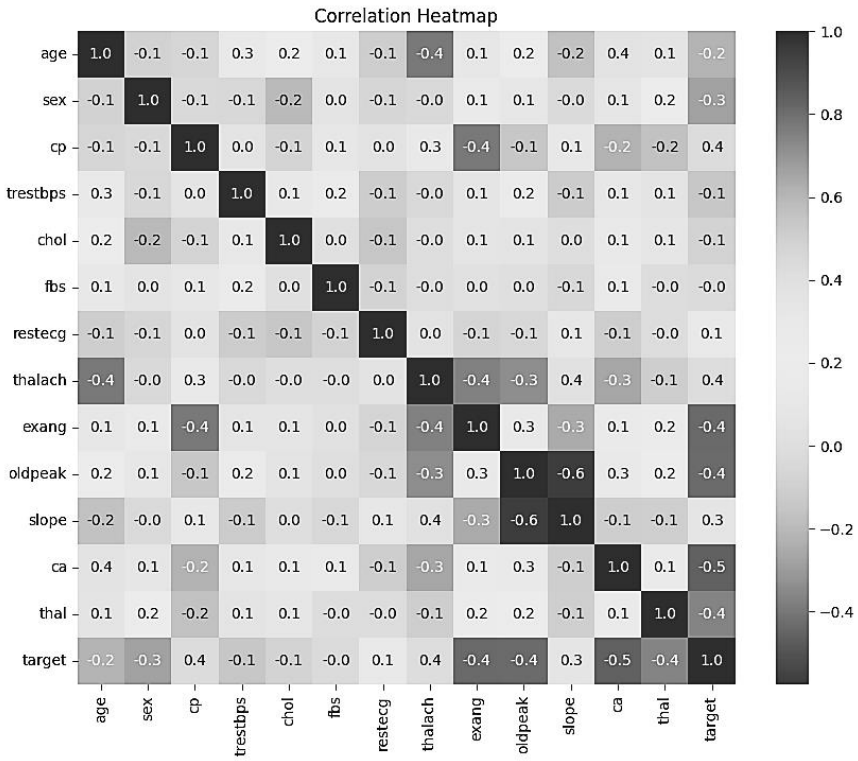

Fig 5-4-3 : Correlation Heatmap

Feature	Correlation	Interpretation
cp (chest pain type)	+0.43	Higher chest pain type associated with disease
thalach (max rate)	+0.42	Higher heart rate linked to disease presence
slope (ST slope)	+0.35	ST segment slope related to disease
exang (exr. angina)	−0.44	Strong negative relationship with target
oldpeak (ST-Depr.)	−0.43	Higher ST depression linked to disease
ca (major vessels)	−0.39	More vessels associated with disease
thal (thalassemia)	−0.34	Certain thal types linked to disease
sex	−0.28	Male shows negative correlation with target
age	−0.23	Weak negative relationship
chol (cholesterol)	+0.09	Very weak correlation
fbs (fasting blood sugar)	+0.03	Negligible relationship

- Cholesterol and fasting blood sugar show almost no correlation with target.
- Not irrelevant clinically but add little predictive information among features.
- Confirmed later by L1 regularization zeroing both features.

6. FEATURE ENGINEERING

6.1 Feature-Target Separation

- Feature matrix X: 302 × 13 (all predictor columns)
- Target vector y: 302 × 1 (binary target column)

6.2 Train-Test Split

Parameter	Value	Reason
Test size	20% (61 samples)	Standard split ratio
Training size	80% (241 samples)	Remaining data for model training
random_state	42	Ensures reproducibility
stratify	y	Maintains class balance (~54/46)

- Without stratification, random sampling could produce unrepresentative split on a small dataset

6.3 Feature Standardization

$$z = \frac{x - \mu}{\sigma}$$

- Applied using scikit-learn's StandardScaler
- Transforms each feature to zero mean, unit variance

Why necessary:

- Raw features have vastly different scales (chol: 126–564 vs sex: 0–1)
- Without scaling: large-scale features dominate gradient updates
- After scaling: all contribute equally → faster convergence, balanced weights

Preventing data leakage:

1. ***fit_transform()*** on training data - computes μ and σ from training data, applies transformation
2. ***transform()*** on test data - applies same μ and σ from step 1, does NOT recompute

- If fit on entire dataset before splitting → test distribution leaks into training → optimistically biased results

7. MODEL DEVELOPMENT

7.1 Logistic Regression

7.1.1 Hyperparameter Tuning with GridSearchCV

- Exhaustive search over parameter grid with k-fold cross-validation

Parameter grid:

Parameter	Values Tested	Purpose
C	[0.01, 0.1, 1, 10]	Inverse regularization strength (C = 1/λ)
penalty	['l1', 'l2']	Regularization type
solver	['liblinear']	Supports L1 and L2

- 4 × 2 × 1 = 8 combinations × 5 folds = 40 total models fit

5-fold cross-validation process:

1. Training data (241 samples) divided into 5 folds (~48 each)
2. For each parameter combination: each fold is held out for validation once; remaining 4 used for training
3. Validation scores averaged → single performance estimate per combination
4. Every data point used for training and validation exactly once → reduces overfitting risk

Best parameters: C=0.1, penalty=L1, solver=liblinear

- C=0.1 → relatively strong regularization optimal
- L1 over L2 → sparse feature selection more effective than uniform shrinkage

7.1.2 Feature Importance - Coefficient Analysis

- All features standardized → coefficient magnitudes directly comparable
- Larger absolute value → stronger predictive influence

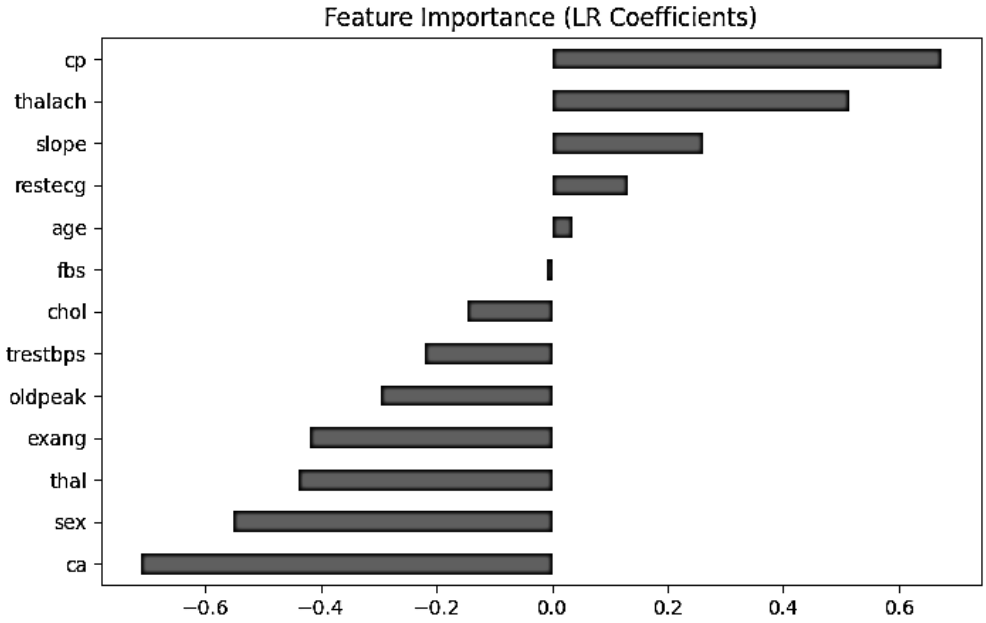

Fig 7-1-1 : Feature Importance [LR]

Feature	**Coefficient**	**Direction**	**Interpretation**
cp	+0.60	Positive	Strongest predictor: chest pain type linked to disease
thalach	+0.38	Positive	Higher max HR associated with disease
slope	+0.32	Positive	ST slope related to disease
sex	+0.31	Positive	Male associated with higher risk
ca	−0.44	Negative	Major vessels linked to disease (encoding effect)
oldpeak	−0.35	Negative	Higher ST depression associated with disease
exang	−0.34	Negative	Exercise-induced angina linked to disease
thal	−0.28	Negative	Thalassemia type associated with disease
age	0.00	Zero (L1)	Removed by L1 regularization
trestbps	0.00	Zero (L1)	Removed
chol	0.00	Zero (L1)	Removed; weak correlation
fbs	0.00	Zero (L1)	Removed; near-zero correlation
restecg	0.00	Zero (L1)	Removed

Coefficient interpretation:

- Positive Coeff. → increases log-odds of disease; negative → decreases
- cp = +0.60 → one-SD increase in chest pain type multiplied odds by $e^{0.60} \sim 1.82$

Zeroed features - notable finding:

- Cholesterol widely recognized as CVD risk factor in general medical practice
- In this dataset (with 12 other features available), cholesterol adds no predictive value
- Its information is likely captured/superseded by exercise test results (cp, thalach, oldpeak)
- Consistent with correlation analysis: chol showed only r = 0.09 with target

7.2 Artificial Neural Network

7.2.1 Architecture Design

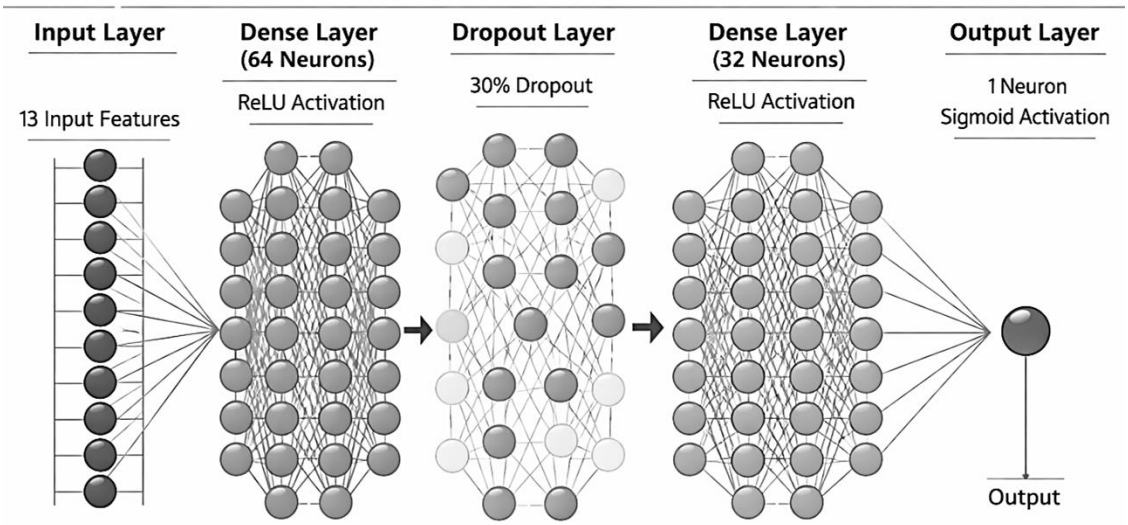

Fig 7-2-1 : ANN Design

- **Parameter count**: 64×13+64 + 32×64+32 + 1×32+1 = 3,009 trainable parameters
- Narrowing (64→32→1) → progressive compression of representations → feature extraction
- Shallow (2 hidden layers) and narrow - intentional for 302-sample dataset
- Deeper/wider → more parameters relative to samples → overfitting risk
- ReLU in hidden layers → non-linearity; Sigmoid at output → probability

7.2.2 Training Configuration

Parameter	Value	Rationale
Optimizer	Adam	Adaptive learning rate; robust optimizer
Loss	Binary cross-entropy	Standard for binary classification
Epochs	50	Enough for convergence on small dataset
Batch size	16	Small batches add regularization noise
Validation split	0.2	20% of training (~48 samples) for validation

7.2.3 Training Progress

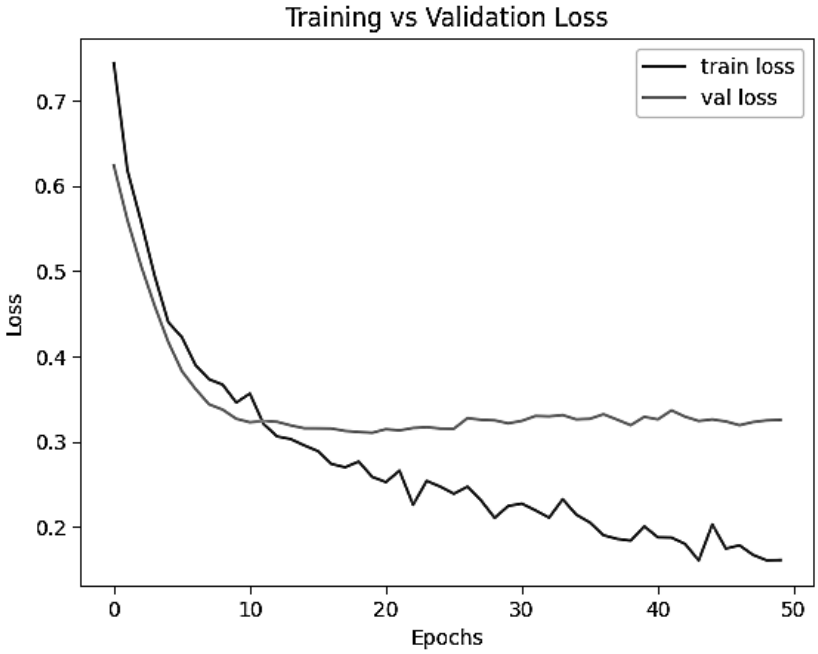

Fig 7-2-3 : Train Vs Validation

- Both curves decrease together and remain close → generalizable learning, not memorization
- No train-Val divergence observed → Dropout and moderate architecture providing adequate regularization

8. MODEL EVALUATION AND VALIDATION

8.1 Cross-Validation Strategy

- **LR**: 5-fold CV via GridSearchCV → dual purpose: hyperparameter selection + model validation → final evaluation on held-out test set (61 samples)
- **ANN**: 20% validation split during training → ~48 samples held out per epoch for overfitting detection

8.2 Logistic Regression Results

Evaluated on 61-sample test set (C=0.1, L1, liblinear):

Metric	Value
Accuracy	0.803
Precision	0.800
Recall	0.848
F1-score	0.824
AUC	0.892

Confusion Matrix:

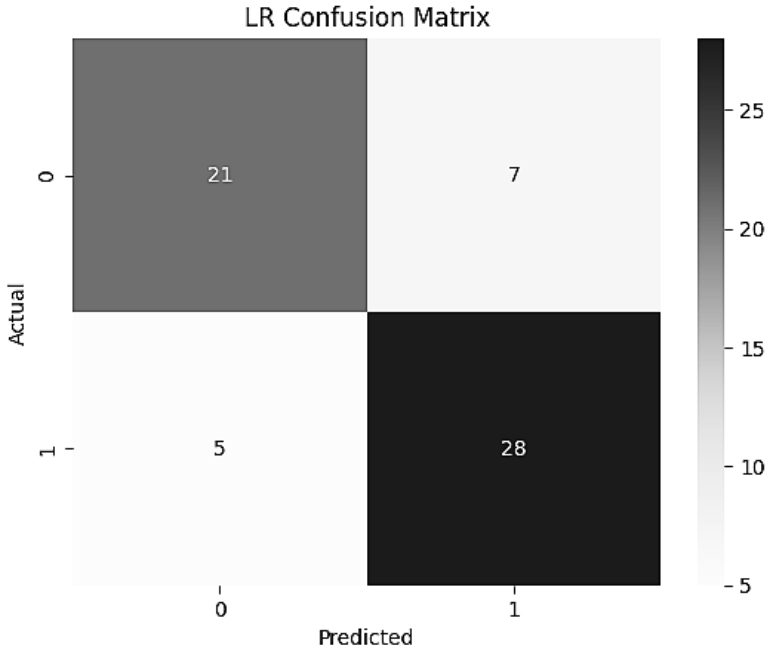

Fig 8-2 : LR Confusion Matrix

Interpretation:

- 49/61 (80.3%) correctly classified
- 28/33 disease cases caught (84.8% recall) - majority of disease patients detected
- 5 FN - disease patients misclassified as healthy → serious clinical error (missed referral)
- 7 FP - healthy patients flagged → unnecessary follow-up (inconvenient, not dangerous)

8.3 ANN Results

Evaluated on same 61-sample test set:

Metric	Value
Accuracy	0.787
Precision	0.812
Recall	0.788
F1-score	0.8
AUC	0.839

Confusion Matrix:

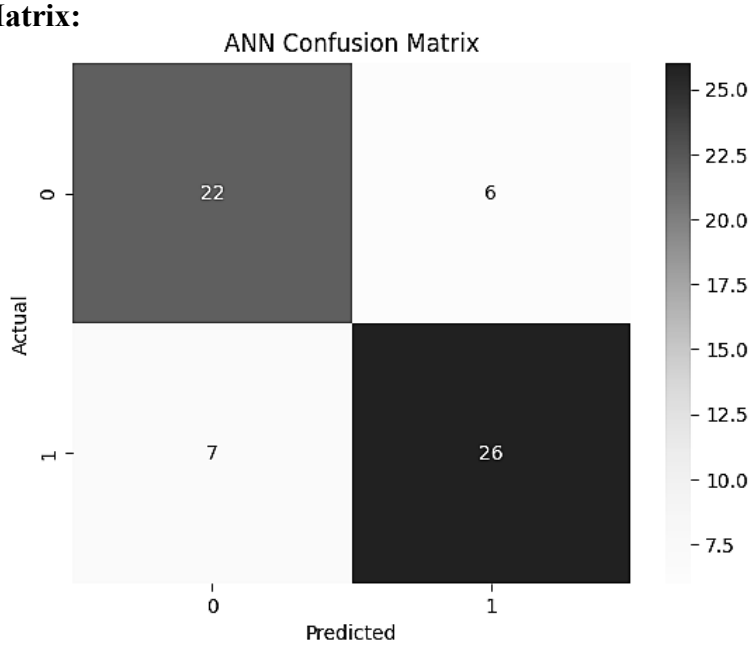

Fig 8-3 : ANN Confusion Matric

Comparison with LR:

- ANN missed 7 disease patients vs LR's 5
- Entire performance difference = 1 additional false negative

9. RESULTS INTERPRETATION

9.1 Model Comparison

Metric	Logistic Regression	ANN	Winner
Accuracy	**80.3%**	78.7%	LR (+1.6%)
Precision	80.0%	**81.2%**	ANN (+1.2%)
Recall	**84.8%**	78.8%	LR (+6.0%)
F1-Score	**82.4%**	80.0%	LR (+2.4%)
AUC	**0.86**	0.84	LR (+0.02)

- LR wins 4/5 metrics; ANN wins only precision by marginal 1.2%
- Most significant: recall difference of 6 percentage points → LR catches 6% more disease cases → clinically meaningful

9.2 ROC Curve Analysis

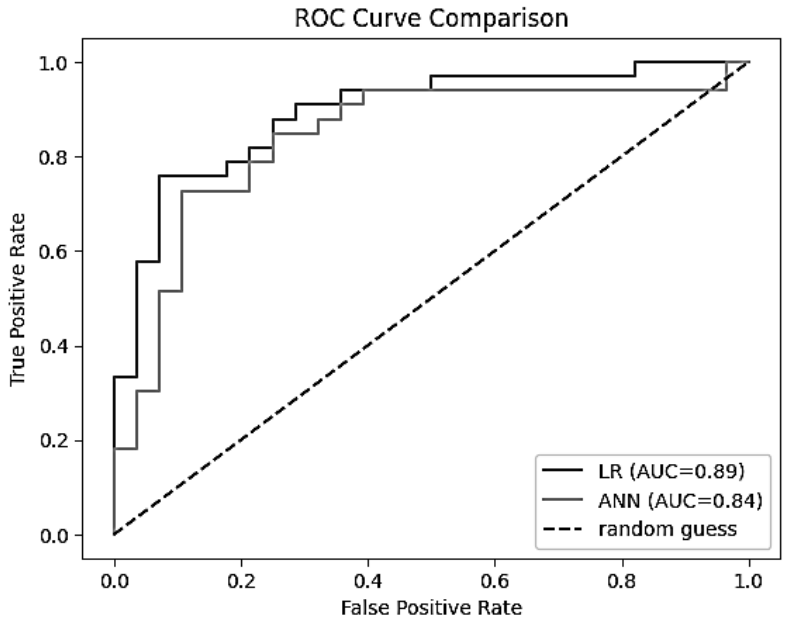

Fig 9-2 : ROC Curve Comparison

- Diagonal = random guessing (AUC = 0.5) | Both models substantially above diagonal
- LR curve (AUC=0.86) sits slightly above ANN curve (AUC=0.84) across most thresholds
- LR provides better class discrimination at nearly all operating points

9.3 Why Logistic Regression Outperformed the ANN

1. Insufficient data for neural network learning

- ANN: ~3,000 parameters, only ~193 effective training samples (241 minus 20% Val split)
- General guideline: NNs need >=10× parameters in samples → ~30,000 needed for this architecture
- Not enough data to learn patterns beyond what a simpler model already captures

2. Low-dimensional, approximately linear relationships

- Only 13 features; feature-target correlations are approximately linear (Section 5.4.3)
- ANNs excel at complex non-linear interactions in high-dimensional data → unnecessary here
- LR assumes linear feature-to-log-odds relationship → well-matched to actual data

3. Effective L1 feature selection

- L1 removed 5 irrelevant features → effective dimension reduced from 13 to 8
- Reduces noise, focuses on informative signals
- ANN has no equivalent built-in feature selection → must learn to ignore irrelevant features from data alone → requires more samples

4. Interpretability advantage

- LR coefficients provide direct insight into prediction logic → valid-able by domain experts
- ANN = "black box" → comparable accuracy but cannot explain individual predictions

10. CONCLUSION

10.1 Summary of Findings

- **Both models achieved ~80% accuracy** - consistent with published literature and community results
- **LR slightly but consistently outperformed ANN (80.3% vs 78.7%)** - won 4/5 metrics; most significant: recall (84.8% vs 78.8%)
- **Most predictive features**: cp (+0.60), thalach (+0.38), ca (−0.44)
- **L1 regularization performed automatic feature selection** - zeroed out age, trestbps, chol, fbs, restecg → exercise test results supersede routine measurements
- **Model complexity ≠ superior performance** - with 302 samples and 13 features, LR (8 effective parameters) outperformed ANN (3,009 parameters)

10.2 Limitations

- **Small dataset (302 records)**: test set of 61 → each prediction significantly impacts metrics; results may not be stable across splits
- **Single-source data**: Cleveland Clinic only → performance on other regions/demographics unknown
- **Non-deterministic ANN**: results vary between runs (random init, dropout, batch order) → reported results represent one run
- **Cross-sectional only**: single snapshot per patient, no follow-up/temporal patterns
- **Binary classification only**: original dataset has severity 0–4; this project uses binary (0 vs 1–4) → loses granularity

10.3 Future Work

- **Larger datasets**: combine all 4 UCI hospitals (920 records) or other public datasets → potentially favor ANN
- **Ensemble methods**: Random Forest, XGBoost, Gradient Boosting → strong on tabular data
- **Better ANN**: early stopping, learning rate scheduling, batch normalization (with more data)
- **Clinical deployment**: Flask/Streamlit web app with appropriate disclaimers

10.4 Ethical Considerations

Patient Privacy:

- UCI dataset is public and anonymized
- Real-world deployment → sensitive health data → requires consent, encryption, access controls, audit trails

Dataset Bias:

- Single institution (Cleveland Clinic, USA) → single demographic profile
- May not generalize across ethnicities, ages, or regions
- Multi-center validation essential before clinical use

Model as Assistive Tool:

- ~80% accuracy → incorrect for ~1 in 5 patients
- Must be positioned as first-pass screening only - not standalone diagnosis
- Never a replacement for clinical judgment

Fairness:

- Sex used as predictive feature → reflects real physiological differences in CVD presentation
- Deployment must be transparent about demographic influence on predictions
- Must not introduce or amplify disparities in healthcare access

References

[1] World Health Organization. (2024). *Cardiovascular diseases (CVDs).* [Online]. Available: https://who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)

[2] S. Mohan, C. Thirumalai, and G. Srivastava. (2019). Effective heart disease prediction using hybrid machine learning techniques. *IEEE Access*, 7, 81542–81554. https://doi.org/10.1109/ACCESS.2019.2923707

[3] M. M. Ali et al. (2021). Heart disease prediction using supervised machine learning algorithms: Performance analysis and comparison. *Computers in Biology and Medicine*, 136, 104672. https://doi.org/10.1016/j.compbiomed.2021.104672

[4] UCI Machine Learning Repository. (n.d.). *Heart disease dataset*. [Online]. Available: https://archive.ics.uci.edu/dataset/45/heart+disease

[5] D. Kumar Pravan. (2019). *Heart disease UCI* [Dataset]. Kaggle. https://www.kaggle.com/code/mragpavank/heart-disease-uci

[6] K. Naik. (2020, Aug. 15). *Heart disease classification* [Video]. YouTube. https://www.youtube.com/watch?v=75OJvlhFUMY

[7] StatQuest with J. Starmer. (2018, Jul. 2). *Logistic regression details pt 1: Coefficients* [Video]. YouTube. https://www.youtube.com/watch?v=yIYKR4sgzI8

[8] 3Blue1Brown. (2017). *Neural networks* [Video playlist]. YouTube. https://youtube.com/playlist?list=PLZHQObOWTQDNU6R1_67000Dx_ZCJB-3pi

[9] scikit-learn developers. (n.d.). *sklearn.linear_model.LogisticRegression*. scikit-learn. [Online]. Available: https://scikitlearn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[10] Keras Team, *Structured data classification from scratch*. [Online]. Available: https://keras.io/examples/structured_data/structured_data_classification_from_scratch/